

TIREx Tracker: The Information Retrieval Experiment Tracker

Tim Hagen

Maik Fröbe
Matthias Hagen

Jan Heinrich Merker
Martin Potthast

Harrisen Scells

IR Evaluation

IR has a strong focus on effectiveness

- ❑ 1966: Cranfield test collection

IR Evaluation

IR has a strong focus on effectiveness

- ❑ 1966: Cranfield test collection

Details are important for effectiveness evaluation

- ❑ Significance testing
- ❑ Multiple testing correction
- ❑ Pros/cons of different metrics

Established file formats and technologies

```
trec_eval -m ndcg qrels.txt run.txt
```

IR Evaluation

IR has a strong focus on effectiveness

- ❑ 1966: Cranfield test collection

Details are important for effectiveness evaluation

- ❑ Significance testing
- ❑ Multiple testing correction
- ❑ Pros/cons of different metrics

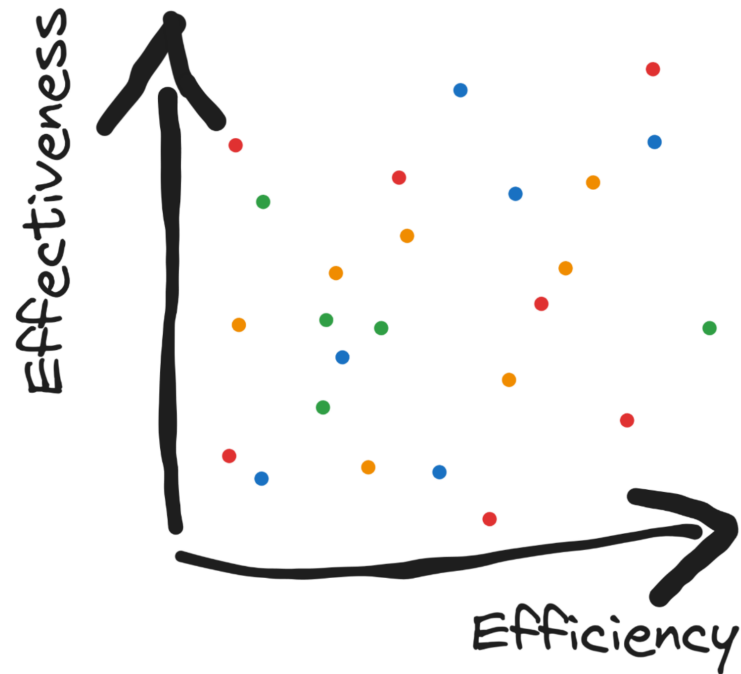
Established file formats and technologies

```
trec_eval -m ndcg qrels.txt run.txt
```

Efficiency evaluation often missing

→ ReNeuIR aims to improve efficiency evaluation in IR

Enabling A Holistic Picture of Efficiency and Effectiveness



- ❑ We “outsource” effectiveness to others
- ❑ We want to enable as many interpretations of efficiency as possible

Measuring Efficiency

Possible metrics for efficiency:

- ❑ Runtime (wallclock, system or user time?)
- ❑ CPU/GPU utilization (higher is better?)
- ❑ RAM usage (lower is better?)
- ❑ Energy consumption
- ❑ Estimated emissions in CO₂eq

Ideally, we want to have something similar to `trec_eval` and runfiles that captures all of these:

- ➔ Add runtime information to `ir_metadata`
`ir_metadata` is a file format to store PRIMAD metadata
- ➔ Develop TIREx tracker for creating these files *automatically*
TIREx Tracker collects and stores raw data

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml"):
    # Run IR Experiment
```

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml"):
    # Run IR Experiment
```

Collects PRIMAD metadata:

✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml"):
    # Run IR Experiment
```

Collects PRIMAD metadata:

- ✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries
- ✗ **Research goal** – Venue, publication, metrics, baseline

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml"):
    # Run IR Experiment
```

Collects PRIMAD metadata:

- ✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries
- ✗ **Research goal** – Venue, publication, metrics, baseline
- ✓ **Implementation** – Command, programming language, repository, commit hash

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml") :
    # Run IR Experiment
```

Collects PRIMAD metadata:

- ✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries
- ✗ **Research goal** – Venue, publication, metrics, baseline
- ✓ **Implementation** – Command, programming language, repository, commit hash
- ✗ **Method** – Higher level description of the retrieval components and their parameters

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml") :
    # Run IR Experiment
```

Collects PRIMAD metadata:

- ✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries
- ✗ **Research goal** – Venue, publication, metrics, baseline
- ✓ **Implementation** – Command, programming language, repository, commit hash
- ✗ **Method** – Higher level description of the retrieval components and their parameters
- ✓ **Actor** – Researcher information: Name, ORCID, team, ...

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml") :
    # Run IR Experiment
```

Collects PRIMAD metadata:

- ✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries
- ✗ **Research goal** – Venue, publication, metrics, baseline
- ✓ **Implementation** – Command, programming language, repository, commit hash
- ✗ **Method** – Higher level description of the retrieval components and their parameters
- ✓ **Actor** – Researcher information: Name, ORCID, team, ...
- ✓ **Data** – Training & testdata and other used artifacts

TIREx Tracker for Metadata

Collect and create `ir_metadata` *automatically*.

Python example:

```
from tirex_tracker import tracking
with tracking(export_file_path="ir-metadata.yml") :
    # Run IR Experiment
```

Collects PRIMAD metadata:

- ✓ **Platform** – CPU, RAM, GPU, OS, and installed libraries
- ✗ **Research goal** – Venue, publication, metrics, baseline
- ✓ **Implementation** – Command, programming language, repository, commit hash
- ✗ **Method** – Higher level description of the retrieval components and their parameters
- ✓ **Actor** – Researcher information: Name, ORCID, team, ...
- ✓ **Data** – Training & testdata and other used artifacts

And efficiency metrics – CPU/GPU/RAM/VRAM usage and runtime

TIRA/TIREx + TIREx Tracker

- ❑ TIRA: Shared task platform [tira.io]

 TIRA

Documentation Forum [Sign Up](#) [Log In](#)  

[TIRA](#) / [TASKS](#) / [IR-BENCHMARKS](#)

ir-benchmarks

by [Webis](#) ([contact](#))

A collection of IR benchmarks on 15 corpora (1.9 billion documents) with 32 well-known shared tasks.

 2022



 40

 0

SUBMIT

TASK WEBSITE

DOCUMENTATION

Submissions

Dataset

antique-test

☐	Team	Approach	Run	nDCG@10 ↓	RR	P@10	 	▼
☐	  f/schlatt	sparse-cross-encoder-4-512	2024-01-04-20-01-54	0.54	0.906	0.639	 	▼
☐	  tira-ir-starter	DLH (tira-ir-starter-pyterrier)	2023-01-07-19-20-25	0.537	0.929	0.754	 	▼
☐	  tira-ir-starter	MonoT5 3b (tira-ir-starter-gygaggle)	2023-02-10-23-28-33	0.537	0.875	0.546	 	▼
☐	  tira-ir-starter	MonoT5 3b (tira-ir-starter-gygaggle)	2023-02-10-23-28-33	0.537	0.875	0.546	 	▼
☐	  tira-ir-starter	DuoT5 base-10k-ms-marco Top-25 (tira-ir-starter-pyterrier)	2023-02-16-17-21-50	0.531	0.898	0.675	 	▼

Containerized submissions that are evaluated without internet access

- ❑ TIREx: Integration of IR into TIRA [[Fröbe et al., 2023](#)]

TIRA/TIREx + TIREx Tracker

- ❑ TIRA: Shared task platform [\[tira.io\]](https://tira.io)
Containerized submissions that are evaluated without internet access
- ❑ TIREx: Integration of IR into TIRA [\[Fröbe et al., 2023\]](#)

Future Work

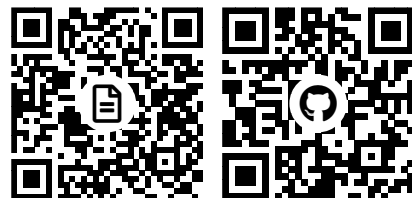
- ❑ Integrate TIREx Tracker into TIREx
 - ➔ Automatic efficiency tracking
 - ➔ Highlight the efficiency-effectiveness trade-off
No single best model but multiple pareto efficient ones
- ❑ Automatically reproduce experiments from their `ir_metadata` file

TIRA/TIREx + TIREx Tracker

- ❑ TIRA: Shared task platform [\[tira.io\]](https://tira.io)

Containerized submissions that are evaluated without internet access

- ❑ TIREx: Integration of IR into TIRA [\[Fröbe et al., 2023\]](#)



Future Work

- ❑ Integrate TIREx Tracker into TIREx

➔ Automatic efficiency tracking

➔ Highlight the efficiency-effectiveness trade-off

No single best model but multiple pareto efficient ones

- ❑ Automatically reproduce experiments from their `ir_metadata` file

Thank you for your attention!